```
  def add(ingredient)
    ingredients << ingredient
  end

  def price
    1.00
  end
end
```

When you run your specs, you'll see one dot for each completed example, with failures and exceptions called out with letters:

```
$ rspec
.F

Failures:

  1) A cup of coffee with milk costs $1.25
     Failure/Error: expect(coffee.price).to eq(1.25)

       expected: 1.25
            got: 1.0

       (compared using ==)
     # ./spec/coffee_spec.rb:26:in `block (3 levels) in <top (required)>'

Finished in 0.01222 seconds (files took 0.08094 seconds to load)
2 examples, 1 failure

Failed examples:

rspec ./spec/coffee_spec.rb:25 # A cup of coffee with milk costs $1.25
```

Here, we see one dot for the passing example, and one `F` for the failure. This format is good for showing the progress of your specs as they execute. When you've got hundreds of examples, you'll see a row of dots marching across the screen.

On the other hand, this output doesn't give any indication of which example is currently running, or what the expected behavior is.

When you need more detail in your test report, or need a specific format such as HTML, RSpec's got you covered. By choosing a different *formatter*, you can tailor the output to your needs.

A formatter receives events from RSpec—such as when a test fails—and then reports the results. Under the hood, it's just a plain Ruby object. You can easily create your own, and in *How Formatters Work*, on page 153 you'll see how to do that. Formatters can write data in any format, and send the output anywhere (such as to the console, a file, or over a network). Let's take a look at another one of the formatters that ships with RSpec.